

Adaptive inference for remote sensing scene classification at the edge

Confidence-guided vision transformers in constrained-resource environments

Luka Waronig

University of Amsterdam

[luka.waronig@student.uva.nl]

Shubh Gupta

University of Amsterdam

[shubh.gupta@student.uva.nl]

October 10, 2026

Abstract

Abstract

Conventional vision transformers execute the same computational path for every image, regardless of how much processing is needed to obtain a reliable prediction. This uniformity becomes particularly costly at edge deployments, where computational resources are limited and may change over time. This paper investigate whether a compact vision transformer can adapt its inference depth to both image difficulty and resource availability. Starting from a fine-tuned DeiT-Tiny classifier on NWPU-RESISC45, we develop a shared-weight architecture with intermediate prediction exits trained through supervised learning and knowledge distillation. A calibrated confidence policy determines when inference can stop, while an explicit compute allowance constrains the maximum execution depth. Experimental results show that confidence-guided execution substantially reduces average computation while preserving classification accuracy close to the full-depth reference. At matched computational cost, it also outperforms input-independent depth allocation. These findings demonstrate how adaptive computation can make remote sensing scene classification more efficient, while revealing the trade-offs that arise when computational resources become constrained.

1 Introduction

Remote sensing classifiers are increasingly deployed on devices that do not always have the same compute budget for every incoming image. A field gateway, for example, may need to classify overhead image tiles while also handling ingestion, caching, communication, or other concurrent tasks. In such settings, executing the full network for every image can be wasteful: some images are easy and can be classified reliably after limited processing, while others benefit from deeper computation.

Adaptive inference offers a direct way to exploit this difference. Instead of executing a fixed number of layers for every input, the model can stop early when an intermediate prediction is already reliable. This produces multiple operating points from one shared model and makes it possible to react to external resource constraints at runtime.

We investigate whether a compact vision transformer can allocate computation according to both image difficulty and the available resource budget. Rather than comparing model architectures, the study holds the transformer backbone fixed and examines the value of changing its execution depth.

Our contributions are threefold. First, we train a shared-weight vision transformer with intermediate classifiers and knowledge distillation. Second, we develop a confidence-based stopping policy

that operates within an explicit compute allowance. Third, we evaluate the resulting accuracy–computation trade-off against static execution and input-independent allocation, supported by runtime latency measurements.

2 Related Work

2.1 Remote sensing scene classification

NWPU-RESISC45 was introduced to provide a challenging benchmark for remote sensing scene recognition, including substantial variation within classes and visual similarity between classes [1]. Subsequent studies have compared different deep-learning architectures and pretraining strategies on this task. In particular, the AiTLAS benchmark evaluates convolutional networks, vision transformers, and other architectures across multiple Earth-observation datasets [8].

These results motivate the use of a pretrained vision transformer as a suitable classification backbone. Our objective, however, is not to establish which architecture achieves the highest classification accuracy. We instead investigate how computation can be allocated within one compact model.

2.2 Knowledge distillation and early exiting

Knowledge distillation transfers information from a trained teacher to a student model, commonly by matching softened prediction distributions [4]. DeiT demonstrates the relevance of data-efficient transformer training and distillation for image classification [3].

Early-exit architectures extend this idea by attaching classifiers to intermediate representations. LGViT investigates early exiting in vision transformers and identifies weak shallow representations as an important challenge. Its training method uses additional supervision and distillation to improve intermediate predictions [9].

More recently, EnViT examines interference between early and late exits during shared-weight training and proposes an exit-aware method to improve this trade-off [7].

2.3 Adaptive inference for remote sensing

Zhao et al. propose E3C, combining knowledge distillation and early exiting for remote sensing scene classification on resource-constrained devices [6]. Their work demonstrates that classification accuracy must be considered alongside inference latency and energy consumption.

Our implementation investigates a related question using a compact DeiT-Tiny backbone with jointly trained intermediate exits. We focus on the contribution of confidence-guided execution, compare it with fixed-depth and random allocation, and examine how the same model responds to changing compute allowances.

3 Problem setting and dataset

3.1 Deployment scenario

The intended deployment is a resource-constrained field gateway that receives preprocessed overhead image tiles and assigns scene labels for local indexing and retrieval. The gateway may not always be able to afford the same amount of computation. We therefore study a model that can either:

1. stop early when it is already confident; or
2. stop because the current resource allowance forbids deeper execution.

Each image is classified independently. The task is not temporal change detection. The output of the adaptive system consists of a scene label, a confidence score, and the executed exit depth.

3.2 Dataset

We use NWPU-RESISC45, a standard aerial imagery benchmark containing 31,500 RGB images of size 256×256 , with 700 images in each of 45 scene classes [1]. Following the published split protocol used in representation-learning work on this dataset [2], we use:

- 18,900 training images,
- 6,300 validation images,
- 6,300 test images.



Figure 1: Representative images from twelve scene categories in NWPU-RESISC45 [1]. The dataset includes natural landscapes, transport infrastructure, agricultural patterns, and built environments. Visually related categories, such as churches and palaces or different residential densities, illustrate the challenges of scene classification.

Images are resized to 224×224 and normalized using the pretrained DeiT input convention. Training augmentation uses horizontal flips, vertical flips, and right-angle rotations. Evaluation uses deterministic preprocessing.

The 6,300-image validation split is further divided into:

- **model-selection validation** (`val_select`) for model and checkpoint selection,
- **policy validation** (`val_policy`) for confidence calibration and threshold tuning.

The test set is reserved for the final comparison.

4 Methods

4.1 Static baseline

The static reference model is the pretrained `deit_tiny_patch16_224` vision transformer [3], fine-tuned for 45-way scene classification. The model has 12 transformer blocks and a task-specific linear output head.

This baseline always executes all 12 blocks. It provides:

1. the original full-depth accuracy reference,
2. the teacher model for distillation,
3. the initial weights for the adaptive student.

4.2 Adaptive architecture

The adaptive model is initialized from the static baseline and adds two intermediate classification heads after transformer blocks 4 and 8, while retaining the original final output after block 12. The three available exits are therefore:

$$\mathcal{D} = \{4, 8, 12\}.$$

All exits share the same patch embedding and transformer trunk. This means the stored model size does not shrink when an image exits early; instead, the gain comes from skipping later computation for selected inputs.

Table 1: Operating paths of the adaptive model.

Exit	Executed blocks	Role
Exit 4	1–4	Lowest-cost prediction
Exit 8	1–8	Intermediate-cost prediction
Exit 12	1–12	Full-depth prediction

4.3 Training objective

The adaptive student is trained using both class labels and predictions from the frozen static teacher. Let y denote the ground-truth class, and let $z_d \in \mathbb{R}^{45}$ denote the student’s logits at exit $d \in \{4, 8, 12\}$. The training loss is:

$$\begin{aligned} \mathcal{L} = & \text{CE}(z_{12}, y) \\ & + \lambda \cdot \frac{1}{2} \sum_{d \in \{4, 8\}} \text{CE}(z_d, y) \\ & + \beta T_{\text{KD}}^2 \cdot \frac{1}{2} \sum_{d \in \{4, 8\}} D_{\text{KL}}\left(p_{\text{teacher}}^{(T_{\text{KD}})} \parallel p_d^{(T_{\text{KD}})}\right). \end{aligned} \tag{1}$$

Here:

- CE is cross-entropy with the ground-truth label;
- D_{KL} is Kullback–Leibler divergence;
- λ weights the auxiliary supervised loss;
- β weights the distillation term;
- T_{KD} is the distillation temperature.

The softened distributions are defined as

$$p_d^{(T)} = \text{softmax}(z_d/T), \quad p_{\text{teacher}}^{(T)} = \text{softmax}(z_{\text{teacher}}/T). \quad (2)$$

Here, z_{teacher} denotes the frozen teacher’s logits, while z_d denotes the student’s logits at depth d . The divergence is computed from the teacher distribution to the student distribution, and the factor T_{KD}^2 accounts for the temperature-dependent gradient scale. The two intermediate-exit losses are averaged so their combined contribution is independent of the number of early classifiers.

The first term preserves strong full-depth performance. The second makes the intermediate exits directly predictive. The third transfers class-relationship information from the teacher to the cheaper exits [4]. All gradients update only the student; the teacher remains frozen.

4.4 Confidence-based stopping

After training, each exit is temperature-calibrated on validation data [5]. Let T_d denote the calibration temperature for exit d . Confidence is defined as:

$$q_d(x) = \max_c [\text{softmax}(z_d(x)/T_d)]_c. \quad (3)$$

The stopping policy uses thresholds τ_4 and τ_8 :

1. at exit 4, stop if $q_4(x) \geq \tau_4$; otherwise continue if allowed;
2. at exit 8, stop if $q_8(x) \geq \tau_8$; otherwise continue if allowed;
3. exit 12 always returns the final prediction.

Thresholds are selected on the held-out `val_policy` split, not on the test set.

4.5 Resource-aware inference

To model changing resource conditions, we define a maximum allowed exit depth for each image. If the maximum allowed depth is 4, the image must stop at exit 4. If the maximum allowed depth is 8, the model may use exits 4 or 8 but not 12. If the maximum allowed depth is 12, all exits are available.

This is not a physical battery model. It is a controlled compute-allowance experiment that shows how the policy responds when runtime constraints change.

5 Experimental protocol

We compare five kinds of execution:

1. the original static baseline;
2. the adaptive model forced to exit at block 4;
3. the adaptive model forced to exit at block 8;
4. the adaptive model forced to exit at block 12;
5. the adaptive model with the selected confidence-based policy.

For every evaluated configuration, we report:

- classification accuracy,
- GFLOPs per image,
- latency (median and p95, batch size one, CPU),
- exit distribution when relevant.

The main figure is the accuracy–FLOPs plot. We also report a matched-cost random-allocation control to test whether confidence-based routing is genuinely useful, rather than merely producing a cheap mixture of shallow and deep paths.

5.1 Training and model selection

All experiments use the published RESISC45 partition and a fixed random seed of 42. The static DeiT-Tiny model was fine-tuned for 30 epochs using AdamW, a batch size of 32, weight decay of 0.05, and learning rates of 10^{-4} for the pretrained backbone and 5×10^{-4} for the classification head. The schedule used three warm-up epochs followed by cosine decay. The checkpoint with the highest accuracy on `val_select` was retained.

The adaptive student was initialized from the selected static checkpoint. Its two intermediate classifiers each contain layer normalization and a linear classification layer applied to the class token. The student was trained jointly for 15 epochs, using learning rates of 2×10^{-5} for the shared backbone and 2×10^{-4} for the early-exit heads, with AdamW and two warm-up epochs followed by cosine decay.

The distillation settings were $T_{KD} = 3$, $\lambda = 0.5$, and $\beta = 0.5$. The frozen teacher processed the same augmented training images as the student.

The adaptive checkpoint was selected using a weighted validation score:

$$S = 0.2A_4 + 0.3A_8 + 0.5A_{12}, \quad (4)$$

where A_d denotes classification accuracy at exit d on `val_select`. This balances intermediate-exit performance with preservation of full-depth accuracy.

5.2 Confidence calibration and policy selection

The 3,150 images reserved in `val_policy` were divided into 1,565 images for temperature calibration and 1,585 images for threshold selection. Class-stratified sampling was used to construct these subsets.

A separate temperature was fitted for each exit by minimising cross-entropy on the calibration subset. Stopping thresholds were then selected by evaluating candidate policies on the remaining validation images.

The selection rule chose the least expensive policy whose accuracy was within one percentage point of the frozen static teacher on the same policy-selection images. Neither the calibration temperatures nor the thresholds were fitted using test labels.

5.3 Computational and runtime measurements

Inference computation was profiled using `fvcore`, with explicit attention operations and the same counting convention for all configurations. One fused multiply-add was counted as one operation; selected scalar operations were estimated consistently across the execution paths.

The reported FLOPs describe arithmetic work rather than measured energy consumption. Inference latency was also measured in FP32 with batch size one, using one CPU thread and 256 reproducibly sampled test images. Model-forward execution was timed after warm-up; image loading and preprocessing were excluded.

Training was performed locally using the Apple M5 Pro GPU through PyTorch’s MPS backend. The CPU latency measurements characterise this particular hardware and software environment, not a dedicated embedded edge accelerator.

To isolate the contribution of confidence-based routing, we additionally compared the selected policy against 30 random permutations of its exit assignments. Each permutation preserved the exact number of images processed at each depth, and therefore the same average arithmetic cost.

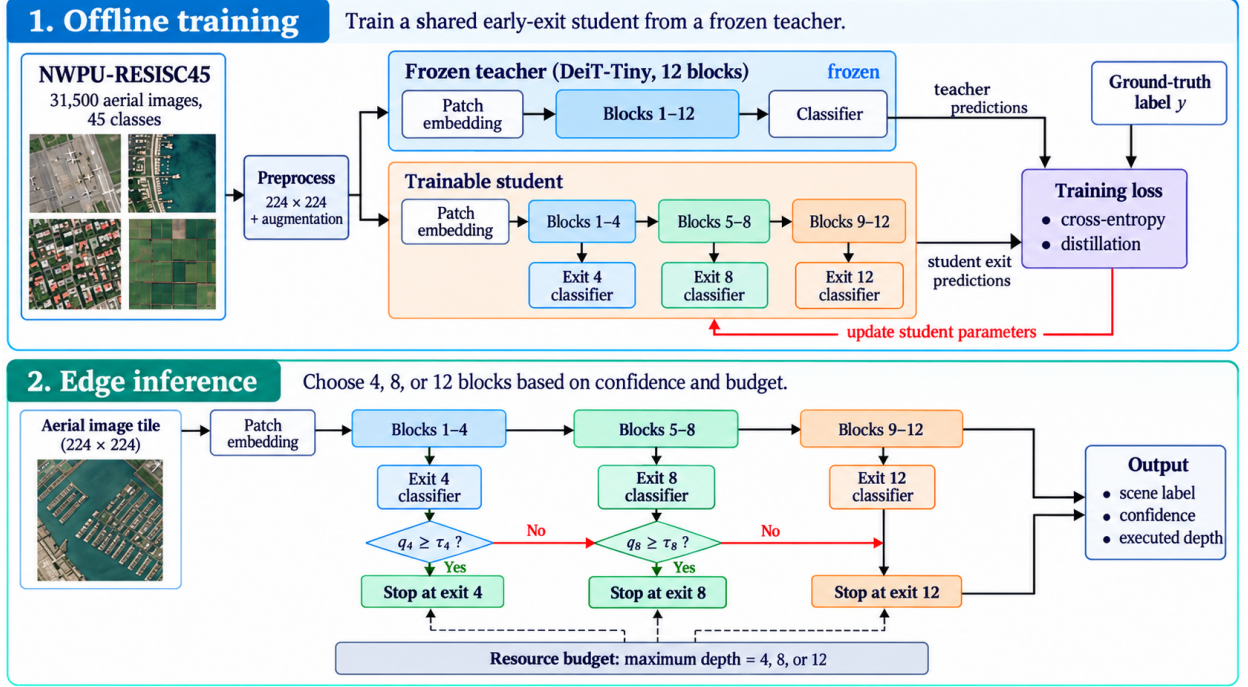


Figure 2: Architecture of the adaptive inference framework. During offline training, a frozen DeiT-Tiny teacher model guides a shared early-exit student through knowledge distillation and supervised classification. During inference, calibrated confidence and the available compute budget determine whether an image exits after 4, 8, or 12 transformer blocks.

6 Results

6.1 Static baseline

The static DeiT-Tiny baseline reached 95.81% accuracy on **val_select**, with the best checkpoint selected at epoch 24. On the held-out test set, it achieved 94.71% accuracy.

Profiling measured 5,533,101 parameters and 1.274 GFLOPs per image for a full-depth forward pass. The most difficult validation classes were *palace*, *church*, *medium_residential*, *dense_residential*, and *industrial_area*, suggesting that the remaining errors came mainly from visually similar scene categories rather than from general instability.

6.2 Adaptive training

The corrected adaptive model jointly fine-tuned the shared transformer and the new exit heads. The selected checkpoint came from epoch 11. Its **val_select** accuracies were:

- 88.03% at exit 4,
- 94.19% at exit 8,
- 95.17% at exit 12.

Compared with the original static baseline, the jointly trained adaptive model lost 0.63 percentage points at full depth. This is expected: the shared backbone now supports multiple exits rather than optimizing only the final one.

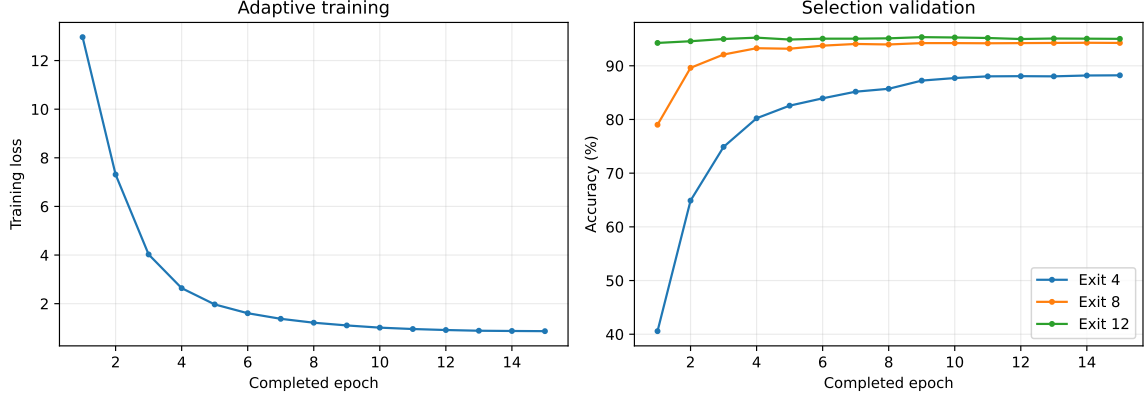


Figure 3: Training of the adaptive variant. The plotted loss is the composite objective in Equation 1. Exit 4 and exit 8 improve rapidly, while exit 12 remains close to the original baseline.

6.3 Main accuracy–efficiency comparison

Table 2 summarizes the main test-set results.

Table 2: Main held-out test results.

Configuration	Accuracy (%)	GFLOPs/image	Median ms	p95 ms
Static baseline	94.71	1.274	5.91	6.11
Adaptive, fixed exit 4	86.56	0.444	2.04	2.26
Adaptive, fixed exit 8	94.06	0.859	3.87	3.93
Adaptive, fixed exit 12	94.21	1.274	5.74	5.80
Adaptive, selected confidence policy	93.79	0.585	2.05	3.98

The selected confidence policy reduced average computation by 54.13% relative to the static baseline:

$$100 \left(1 - \frac{\bar{F}_{\text{adaptive}}}{F_{\text{static}}} \right) = 54.13\%,$$

The corresponding accuracy loss was 0.92 percentage points:

$$94.71 - 93.79 = 0.92.$$

The comparison to fixed exit 8 is also important. The selected policy achieved 93.79% accuracy at 0.585 GFLOPs/image, while fixed exit 8 achieved 94.06% at 0.859 GFLOPs/image. In other words, the adaptive policy lost only 0.27 percentage points relative to fixed exit 8 while using approximately 32% less computation.

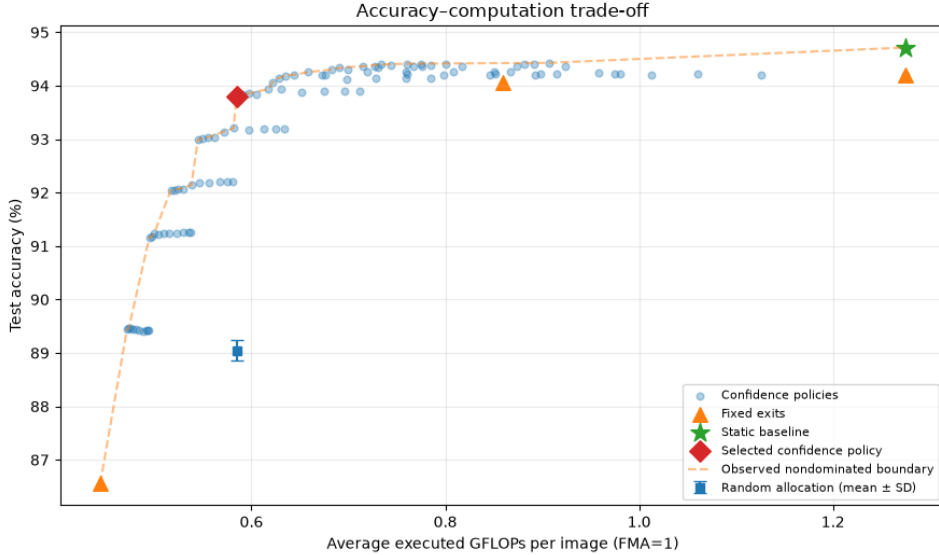


Figure 4: Accuracy-computation trade-off on RESISC45. The selected confidence policy is compared with fixed-depth execution, the original static baseline, and random allocation at matched average computation. The random-allocation error bar shows one standard deviation across 30 permutations. The dashed line denotes the observed nondominated boundary among the evaluated configurations.

6.4 Confidence-based routing versus random routing

To test whether confidence is actually useful, we compared the selected policy to a random-allocation control with the same average compute and the same proportion of 4-, 8-, and 12-block executions.

Table 3: Matched-cost comparison between confidence-guided and random routing.

Routing strategy	Accuracy (%)	GFLOPs/image
Confidence-guided	93.79	0.585
Random allocation	89.05 ± 0.19	0.585

The confidence policy improved accuracy by 4.74 percentage points at exactly the same average computational cost. This shows that the gain does not come merely from mixing shallow and deep paths, but from allocating depth to images more intelligently.

6.5 What deeper blocks contribute

The adaptive model lets us measure how predictions change when we continue from one exit to the next. Across the test set:

- moving from exit 4 to exit 8 corrected 574 previously incorrect predictions but made 101 previously correct predictions wrong;
- moving from exit 8 to exit 12 corrected 118 errors but introduced 109 new errors.

This means the net gain from moving from 4 to 8 blocks is large, while the net gain from moving from 8 to 12 blocks is small. This directly explains why the selected policy rarely needs the final exit.

6.6 Selected policy and exit distribution

The selected policy used calibration temperatures:

$$T_4 = 1.683, \quad T_8 = 1.958, \quad T_{12} = 1.748,$$

and stopping thresholds:

$$\tau_4 = 0.90, \quad \tau_8 = 0.50.$$

Its exit distribution on the full test set was:

- 67.75% of images stopped at exit 4,
- 30.65% stopped at exit 8,
- 1.60% reached exit 12.

The average executed depth is therefore about

$$0.6775(4) + 0.3065(8) + 0.0160(12) \approx 5.35 \text{ blocks.}$$

This is why the adaptive policy can be allowed to use 12 blocks while still executing far fewer than 12 blocks on average.

6.7 Confidence calibration

The stopping policy relies on the confidence of intermediate predictions. We therefore examine the relationship between predicted confidence and observed classification accuracy before and after temperature scaling.

Figure 5 presents reliability diagrams for exits 4 and 8. Perfectly calibrated predictions would lie on the diagonal. Expected calibration error (ECE), computed using ten confidence bins, provides a complementary summary of the difference between confidence and accuracy.

These measurements are used to assess confidence reliability, not to modify the already-selected inference policy. Calibration cannot guarantee the correctness of individual early predictions.

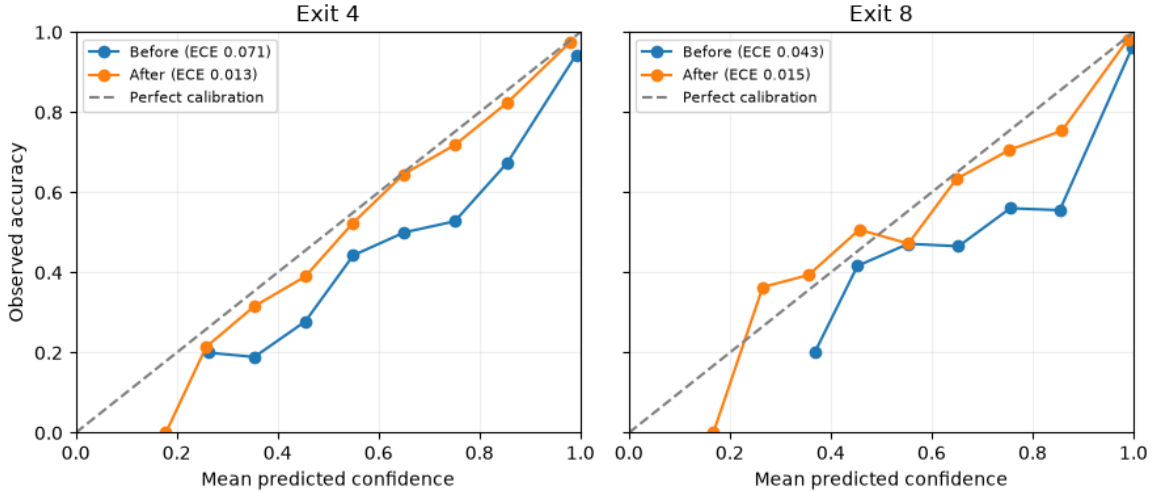


Figure 5: Reliability diagrams for the intermediate classifiers before and after temperature scaling, evaluated on the designated test partition. ECE denotes expected calibration error using ten confidence bins; lower values indicate better calibration under this metric.

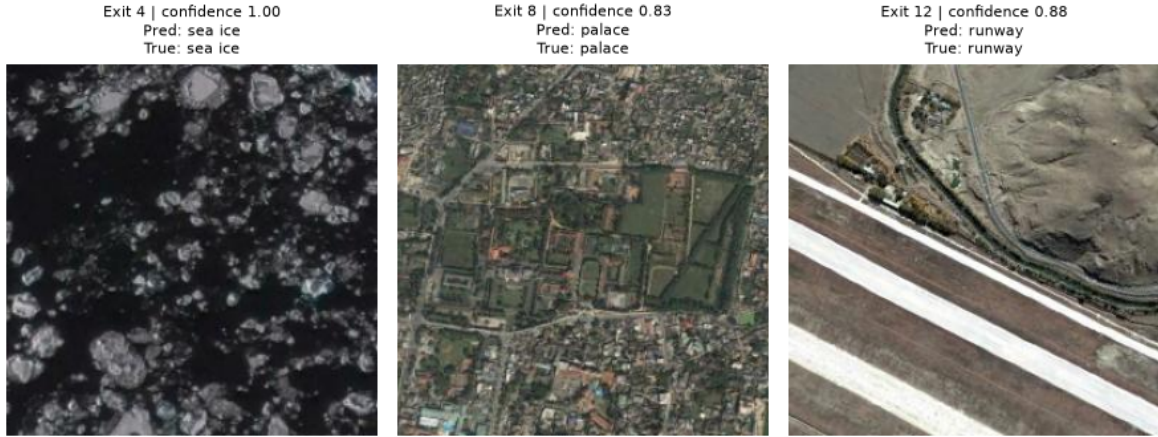


Figure 6: Examples of images processed at different execution depths. One image was sampled reproducibly from each exit group. The figure reports the predicted class, ground-truth class, and calibrated confidence.

6.8 Resource-aware inference

Resource-aware inference asks a different question from the standard accuracy–FLOPs comparison: not just how much computation is useful, but also how the model behaves when some computation is temporarily not available.

Table 4 evaluates the same 6,300 test images under three maximum-depth budgets.

Table 4: Effect of maximum permitted depth on the same held-out test images.

Maximum permitted depth	Accuracy (%)	GFLOPs/image
4 blocks	86.56	0.444
8 blocks	93.62	0.578
12 blocks	93.79	0.585

The difference between the 8-block and 12-block budgets is very small, because only a tiny fraction of images need the final exit under the selected policy. By contrast, forcing all images to stop at 4 blocks produces a much larger accuracy drop.

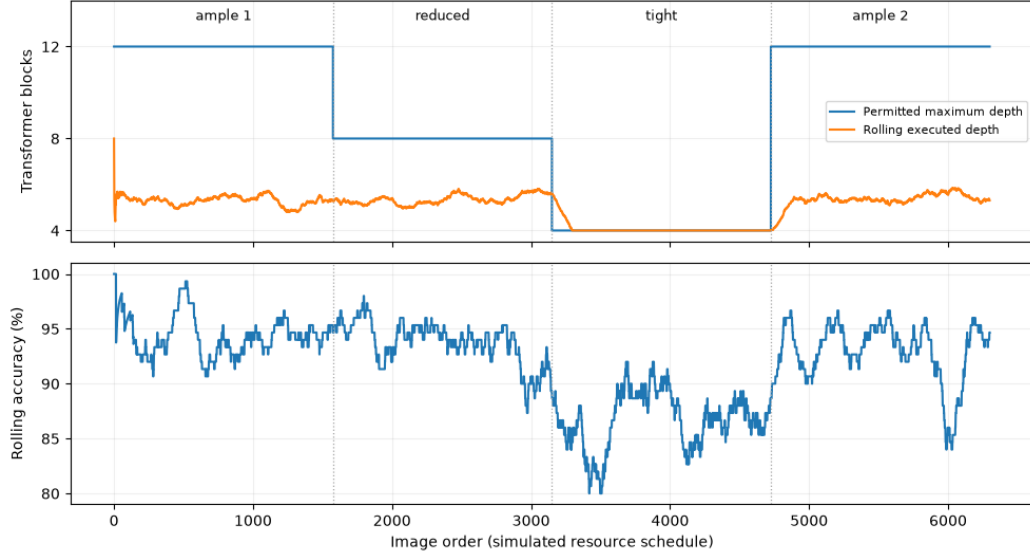


Figure 7: Response to changing compute allowances across a simulated sequence of test images. The upper panel shows the permitted maximum depth and the rolling mean executed depth. The lower panel reports rolling classification accuracy. Rolling statistics use a 150-image window, so their response to a change in resource allowance is not instantaneous.

Figure 7 deserves a short explanation. The permitted maximum depth is the deepest exit the current resource state allows. The rolling executed depth is the average number of transformer blocks actually used over a recent window of images. These two quantities do not need to be equal. For example, when the permitted maximum depth is 12, the model is *allowed* to use all exits, but it often chooses to stop at 4 or 8 blocks because those predictions are already confident. The gap between the two curves is therefore direct evidence of conditional computation.

When the allowed depth is reduced to 8, the model can no longer use exit 12, but this changes little because very few images needed it anyway. The strong change occurs when the maximum depth falls to 4, because then every image is forced to stop early.

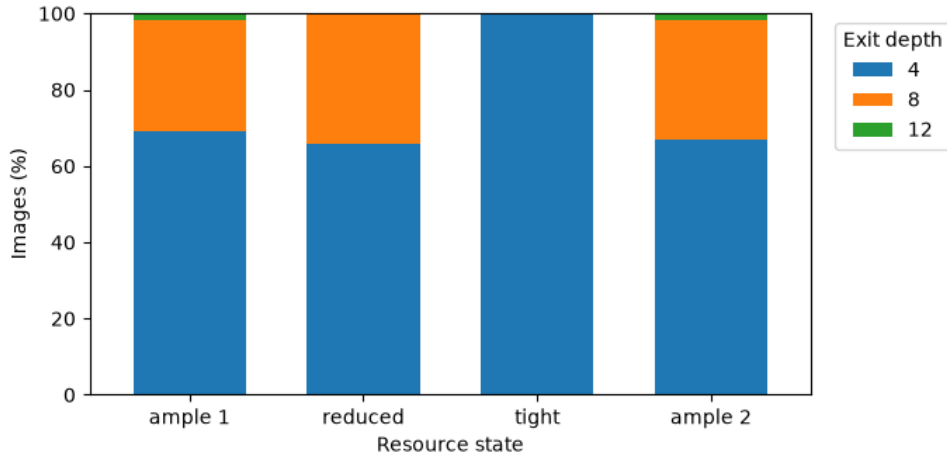


Figure 8: Distribution of executed exit depths under four simulated resource states. Each stacked bar represents the fraction of images processed using 4, 8, or 12 transformer blocks.

The final bar chart in Figure 8 shows how execution decisions change across resource states. It shows what fraction of images exited at each depth. Under a tight budget, every image must exit at block 4. Under ample budgets, the model can choose more freely, and most images still stop at

4, many continue to 8, and only a few reach 12. The chart therefore summarizes how the controller allocates computation under different constraints.

The resource-response phases contain different subsets of images, so their accuracy differences also reflect variation in image difficulty. Table 4 therefore provides the controlled comparison: it evaluates the same images under each permitted maximum depth.

7 Discussion

7.1 When does adaptive computation become useful?

The main finding is that a high-performing image-classification vision transformer does not necessarily need to execute its full computational depth for every input. Under the selected confidence policy, 67.75% of test images exited after four transformer blocks, 30.65% exited after eight, and only 1.60% reached the final classifier.

The average executed depth was approximately 5.35 blocks, despite all twelve blocks being available. This distinction between available computational capacity and computation actually required by the policy is central to adaptive inference. A generous resource budget does not imply that all available computation should be used.

The additional processing was nevertheless valuable for selected images. Moving from exit 4 to exit 8 corrected 574 previously incorrect classifications while introducing 101 new errors. By comparison, moving from exit 8 to exit 12 produced only a small net improvement across the full test partition.

This behaviour helps explain the selected policy. Images classified confidently at shallow depths can avoid later processing, while uncertain images retain access to deeper representations. Confidence-guided execution achieved 4.74 percentage points higher accuracy than random depth allocation at matched computational cost.

The result supports input-dependent computation, but does not establish that confidence always identifies which images would benefit from additional processing. Some confident early predictions remain incorrect, and deeper processing can occasionally replace a correct prediction with an incorrect one.

7.2 Implications for edge deployment

The primary application considered in this study is a resource-constrained field gateway processing aerial image tiles. Such a gateway may perform image ingestion, storage, communication, and classification on the same processor. The resources available for inference may therefore change as other workloads become active.

Our results suggest that adaptive depth could reduce the average computational burden of scene tagging in this setting. Under normal conditions, many incoming images could be classified using shallow exits. When more computation is available, uncertain images could continue through deeper layers. When resources become restricted, the controller can enforce a smaller maximum depth rather than attempting an execution path that exceeds its allowance.

The difference between eight- and twelve-block resource limits is particularly informative. Reducing the maximum depth from twelve to eight lowered test accuracy by only 0.17 percentage points under the selected policy. By contrast, forcing every image to exit after four blocks reduced accuracy to 86.56%. These results suggest that the intermediate configuration provides a useful operating region for this particular model and dataset.

A related application is local processing during aerial surveys. A mobile processor could classify image tiles for indexing or preliminary scene assessment before the complete imagery is transferred

to a larger system. Adaptive inference may be useful when processing capacity must be shared between perception and other tasks.

Onboard Earth-observation processing provides another motivation. ESA’s Φ sat-2 mission demonstrates the broader use of artificial intelligence for image analysis on small satellites, including cloud detection and other onboard processing tasks [10]. Such applications illustrate the value of avoiding unnecessary computation where processing resources and communication opportunities are limited.

These scenarios are potential applications rather than deployments demonstrated by our experiments. RESISC45 contains independently labelled RGB images, not a continuous sensor stream from an operational field gateway or spacecraft. Applying the system to another platform would require testing with its actual imagery, runtime, hardware, and operational constraints.

7.3 Computational savings and runtime behaviour

The selected policy reduced average arithmetic computation by 54.13% compared with the original static baseline. CPU measurements also showed a 65.3% reduction in median inference latency and a 34.9% reduction in p95 latency under the evaluated conditions.

The differences between these percentages are expected. FLOPs measure arithmetic work, whereas execution time also depends on memory traffic, operator implementations, branching, and hardware behaviour.

In our implementation, early exiting avoids executing later transformer blocks and requires no model swapping. However, the complete shared model remains resident in memory. The reduction in executed computation should therefore not be confused with an equivalent reduction in model storage.

Similarly, our experiments do not establish proportional energy savings. A real deployment would require power or energy measurements on the intended device, together with the cost of preprocessing, data movement, and other system operations.

The resource-aware controller also currently receives an externally specified compute allowance. A deployed system would need a mechanism for deriving that allowance from measured operating conditions, such as processor load, available energy, or an inference deadline.

7.4 Limitations and further work

Several limitations define the scope of these findings.

First, jointly training the shared transformer slightly reduced full-depth test accuracy from 94.71% to 94.21%. This may reflect competing optimisation objectives across the exits. More sophisticated training strategies have been proposed to address this problem, including exit-aware methods such as EnViT [7].

Second, the contribution of knowledge distillation was not evaluated independently. The experiments assess the complete distilled early-exit system, not the improvement attributable to distillation alone. A controlled experiment without the distillation term would be necessary to isolate its effect.

Third, the evaluation uses one principal training seed. Repeated training runs would help establish the variability of the reported accuracy–computation trade-off.

Fourth, the RESISC45 evaluation does not establish robustness under geographic, seasonal, sensor, or temporal distribution shift. Confidence calibration may also change under such conditions. A practical implementation would need to consider unfamiliar inputs and changing confidence reliability.

Finally, the designated test partition was inspected during an earlier software-debugging run. The corrected joint training procedure followed the original experimental design, but independent

confirmation on an additional dataset would strengthen the generalisation claim.

These limitations do not remove the measured computational benefit. They identify the additional evidence needed before the method could be considered validated for deployment beyond the benchmark setting.

8 Conclusion

We investigated whether a compact vision transformer can adapt its inference depth to image difficulty and changing computational constraints.

On RESISC45, the selected adaptive policy achieved 93.79% test accuracy while reducing average computation by 54.13% relative to the static baseline, at a cost of 0.92 percentage points in classification accuracy. At matched computation, confidence-guided execution also substantially outperformed input-independent depth allocation.

The results demonstrate that useful intermediate predictions, shared weights, and a lightweight stopping policy can provide multiple operating points within one transformer. Importantly, the system can avoid unnecessary computation even when its full computational capacity is available.

The findings establish a promising accuracy–efficiency trade-off for the evaluated classification task. Further validation on operational imagery and target edge hardware is needed to determine the corresponding benefits in real-world deployments.

Appendix

Static baseline training

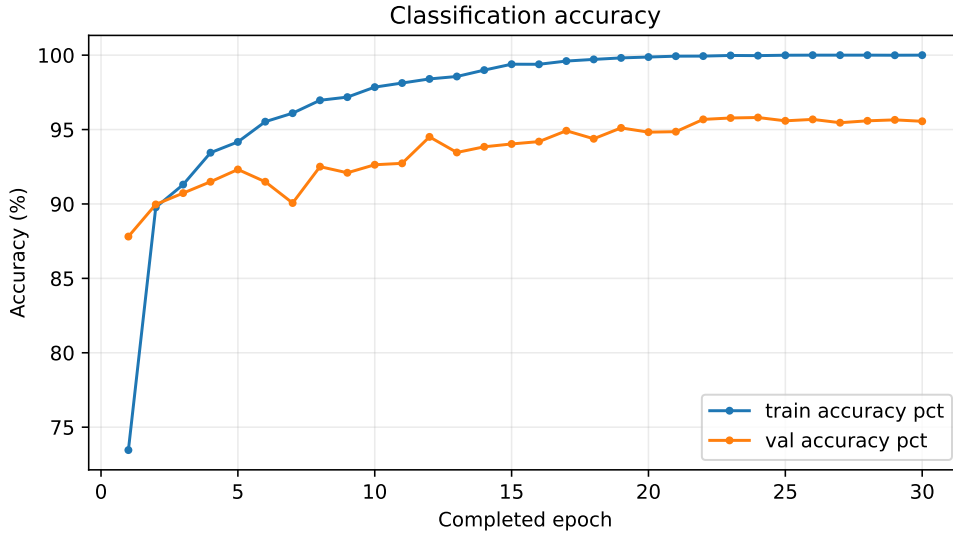


Figure 9: Training and selection-validation accuracy over 30 epochs. The selected static checkpoint corresponds to epoch 24.

References

- [1] G. Cheng, J. Han, and X. Lu. Remote sensing image scene classification: Benchmark and state of the art. *Proceedings of the IEEE*, 105(10):1865–1883, 2017. doi:10.1109/JPROC.2017.2675998.
- [2] M. Neumann, A. Susano Pinto, X. Zhai, and N. Houlsby. In-domain representation learning for remote sensing. *arXiv:1911.06721*, 2019. Preprint.
- [3] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou. Training data-efficient image transformers & distillation through attention. *Proceedings of ICML*, PMLR 139:10347–10357, 2021. Paper.
- [4] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv:1503.02531*, 2015. Preprint.
- [5] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger. On calibration of modern neural networks. *Proceedings of ICML*, PMLR 70:1321–1330, 2017. Paper.
- [6] Y. Zhao, S. Li, and X. Feng. Lightweight remote sensing scene classification on edge devices via knowledge distillation and early-exit. *ACM Multimedia*, 2025. doi:10.1145/3746027.3755609.
- [7] Y. Dong, Q. He, P. Rui, Z. Zheng, Z. Li, F. Chen, H. Jin, and Y. Yang. EnViT: Enhancing the performance of early-exit vision transformers via exit-aware structured dropout-enabled self-distillation. *Proceedings of the AAAI Conference on Artificial Intelligence*, 40(25):20852–20860, 2026. doi:10.1609/aaai.v40i25.39225.
- [8] I. Dimitrovski, I. Kitanovski, D. Kocev, and N. Simidjievski. Current trends in deep learning for Earth observation: An open-source benchmark arena for image classification. *ISPRS Journal of Photogrammetry and Remote Sensing*, 197:18–35, 2023. doi:10.1016/j.isprsjprs.2023.01.014.
- [9] G. Xu, J. Hao, L. Shen, H. Hu, Y. Luo, H. Lin, and J. Shen. LGViT: Dynamic early exiting for accelerating vision transformer. *ACM Multimedia*, 2023. doi:10.1145/3581783.3611762.
- [10] European Space Agency. Φ sat-2: Enhancing onboard AI processing. Mission overview. https://www.esa.int/Applications/Observing_the_Earth/FutureEO/Phsat-2. Accessed October 2026.